Chat History — Dots and Boxes Lab

Chronological record of the discussion about investigating, fixing, testing, and implementing Undo Last Move in the Python Dots and Boxes project.

User

Asked to inspect the README and Python files before changing anything, investigate the Task 1 scoring/turn behavior, reproduce the suspected issue with actual moves, determine whether a real bug existed, identify weaknesses in the game state, and propose a minimal fix without implementing it initially.

Assistant

The investigation found that the original box-completion, scoring, and extra-turn behavior was correct. A 2×2 example showed that a move completing one box correctly increased the current player's score and allowed that player to play again. A separate multi-box/full-game sequence also behaved correctly. The actual weakness was lower-level validation: Board.add_line originally treated any orientation other than H as vertical, so a direct invalid call such as add_line("X", 0, 0) could mutate the board.

User

Asked to implement the proposed fixes while preserving the modular architecture: board.py for board state, rules.py for validation/rules, game.py for game flow, and main.py as the entry point.

Assistant

Implemented stronger board-state validation. Board.can_add_line() validates orientation, coordinates, bounds, and repeated lines. Board.add_line() validates before mutating. rules.py centralizes parsing and validation. game.py preserves the existing scoring rule: completing a box gives the same player another turn.

User

Requested an Undo Last Move feature. Requirements included U/UNDO commands, restoring horizontal and vertical lines, completed boxes, scores, and current player, supporting moves that complete one or multiple boxes, not creating history for invalid input, and giving a clear message when there is nothing to undo.

Assistant

Designed Undo using snapshots stored in Board. A GameSnapshot records horizontal lines, vertical lines, completed boxes, scores, and current player. save_state() stores a copy before a valid move, while undo() restores the most recent snapshot. parse_move() recognizes U and UNDO, and game.py exposes undo_last_move().

User

Asked for an automated unittest suite.

Assistant

Added tests covering horizontal and vertical moves, repeated moves, box completion, adjacent/multiple boxes, turn behavior, complete-board behavior, invalid coordinates, malformed input, undo restoration, empty undo, invalid input not creating history, and U/UNDO parsing. A CHANGES.md was also prepared documenting the investigation, fixes, feature, tests, and design decisions.

User

Ran: python -m unittest discover -s tests -v. The local run reported 20 tests with 2 failures and 6 errors. The errors were because DotsAndBoxes did not have undo_last_move(), and the failures were because parse_move("U") did not return the expected undo command.

Assistant

Explained that this was a code/version mismatch: the tests expected the Undo implementation, while the local production game.py and rules.py were still pre-Undo. The recommended correction was to update board.py, rules.py, and game.py rather than weakening the tests.

User

Asked where exactly to make the corrections.

Assistant

Identified the required changes: add GameSnapshot/history/save_state/undo/has_undo to board.py; make rules.py recognize U/UNDO; add undo_last_move() and save_state() before each valid move in game.py; leave main.py unchanged.

User

Reported: if I click u after completing a box, I am getting nothing undo.

Assistant

Explained that if the final move completes the entire board, the current while-not-complete loop exits before another command can be entered. For an active game, U should work. The key issue was then identified from the supplied game.py: it never called board.save_state() before making a move.

User

Pasted the current game.py and asked whether it was correct.

Assistant

Confirmed that the scoring and turn logic was correct, but the Undo implementation was incomplete. The missing line was self.board.save_state(self.scores, self.current). It must run after the move is validated and immediately before add_line().

Final corrected game.py

```
from board import Board
from rules import valid_move, completed_boxes, parse_move


class DotsAndBoxes:
    def __init__(self):
        self.board = Board()
        self.current = 0
        self.scores = [0, 0]

    def undo_last_move(self):
        restored = self.board.undo()

        if restored is None:
            print("Nothing to undo.")
            return False

        self.scores, self.current = restored
        print("Last move undone.")
        return True

    def run(self):
        print("Dots and Boxes")
        print("Enter moves as H row col or V row col.")
        print("Rows and columns start at 0.")
        print("Example: H 0 1")
        print("Enter U or UNDO to undo the last valid move.")

        while not self.board.is_complete():
            self.board.display(self.scores, self.current)
```

```
        try:
            raw = input(f"Player {self.current + 1}, move: ")
        except (EOFError, KeyboardInterrupt):
            print("\nGame ended.")
            return

        move = parse_move(raw)

        if move is None:
            print("Invalid format. Use H row col or V row col with non-negative integer coordinates.")
            continue

        if move == "UNDO":
            self.undo_last_move()
            continue

        orientation, row, col = move

        if not valid_move(self.board, orientation, row, col):
            print("Invalid or already-used move.")
            continue

        # Save the complete game state BEFORE making the move
        self.board.save_state(self.scores, self.current)

        before = set(self.board.completed)

        if not self.board.add_line(orientation, row, col):
            print("Invalid or already-used move.")
            continue

        newly_completed = completed_boxes(self.board, before)

        if newly_completed:
            self.scores[self.current] += newly_completed
            print(f"Player {self.current + 1} completed {newly_completed} box(es) and plays again.")
        else:
            self.current = 1 - self.current

self.board.display(self.scores, self.current)
print("Game over!")

if self.scores[0] == self.scores[1]:
    print("The game is a draw.")
else:
    winner = 1 if self.scores[0] > self.scores[1] else 2
    print(f"Player {winner} wins!")
```

Final technical note

The corrected game.py fixes the reported Nothing to undo behavior for moves made while the game is still active, assuming board.py contains save_state() and undo(), and rules.py parses U/UNDO. One separate behavior remains: because the loop is while not self.board.is_complete(), if a move finishes the entire board, the loop exits immediately and no additional U command can be entered afterward.